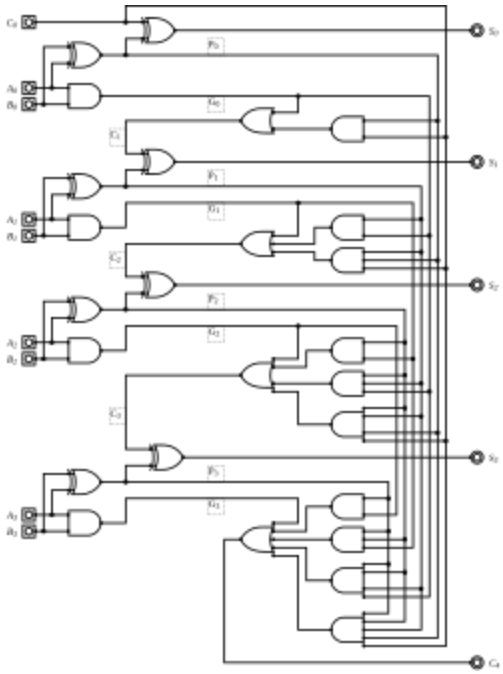
From Wikipedia, the free encyclopedia

"Discrete logic" redirects here. For discrete circuitry, see Discrete circuit. For discrete TTL logic, see Transistor–transistor logic. For the former image processing company, see Discreet Logic.



A logic circuit diagram for a 4-bit carry lookahead binary adder design using only the AND, OR, and XOR logic gates

A **logic gate** is a device that performs a Boolean function, a logical operation performed on one or more binary inputs that produces a single binary output. Depending on the context, the term may refer to an **ideal logic gate**, one that has, for instance, zero rise time and unlimited fan-out, or it may refer to a non-ideal physical device1(https://en.wikipedia.org/wiki/Logic_gate#cite_note-1) (see ideal and real op-amps for comparison).

The primary way of building logic gates uses diodes or transistors acting as electronic switches. Today, most logic gates are made from MOSFETs (metal–oxide–semiconductor field-effect transistors).2(https://en.wikipedia.org/wiki/Logic_gate#cite_note-kanellos-2) They can also be constructed using vacuum tubes, electromagnetic relays with relay logic, fluidic logic, pneumatic logic, optics, molecules, acoustics,3(https://en.wikipedia.org/wiki/Logic_gate#cite_note-3) or even mechanical or thermal4(https://en.wikipedia.org/wiki/Logic_gate#cite_note-4) elements.

Logic gates can be cascaded in the same way that Boolean functions can be composed, allowing the construction of a physical model of all of Boolean logic, and therefore, all of the algorithms and mathematics that can be described with Boolean logic. **Logic circuits** include such devices as multiplexers, registers, arithmetic logic units (ALUs), and computer memory, all

the way up through complete microprocessors,5(https://en.wikipedia.org/wiki/Logic_gate#cite_note-5) which may contain more than 100 million logic gates.

Compound logic gates AND-OR-invert (AOI) and OR-AND-invert (OAI) are often employed in circuit design because their construction using MOSFETs is simpler and more efficient than the sum of the individual gates.6(https://en.wikipedia.org/wiki/Logic_gate#cite_note-6)

There are seven basic logic gates: NOT, OR, NOR (Negation of the OR statement), AND, NAND (Negation of the AND statement), XOR (Exclusive OR), XNOR (Negation of the Exclusive OR statement).7(https://en.wikipedia.org/wiki/Logic_gate#cite_note-7)

History and development

The binary number system was refined by Gottfried Wilhelm Leibniz (published in 1705), influenced by the ancient *I Ching*'s binary system.8(https://en.wikipedia.org/wiki/Logic_gate#cite_note-Nylan2001-8)[[9]](https://en.wikipedia.org/wiki/Logic_gate#cite_note-binary-9) Leibniz established that using the binary system combined the principles of arithmetic and logic.

The analytical engine devised by Charles Babbage in 1837 used mechanical logic gates based on gears.10(https://en.wikipedia.org/wiki/Logic_gate#cite_note-10)

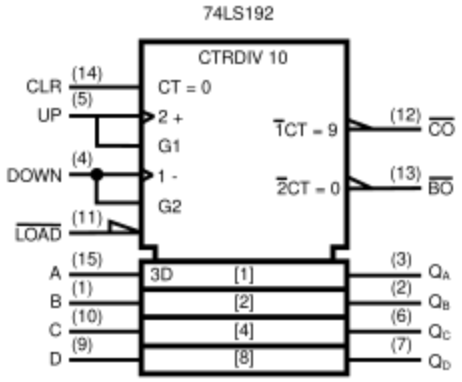
In an 1886 letter, Charles Sanders Peirce described how logical operations could be carried out by electrical switching circuits.11(https://en.wikipedia.org/wiki/Logic_gate#cite_note-P2M-11) Early Electromechanical computers were constructed from switches and relay logic rather than the later innovations of vacuum tubes (thermionic valves) or transistors (from which later electronic computers were constructed). Ludwig Wittgenstein introduced a version of the 16-row truth table as proposition 5.101 of *Tractatus Logico-Philosophicus* (1921). Walther Bothe, inventor of the coincidence circuit,12(https://en.wikipedia.org/wiki/Logic_gate#cite_note-12) got part of the 1954 Nobel Prize in physics, for the first modern electronic AND gate in 1924. Konrad Zuse designed and built electromechanical logic gates for his computer Z1 (from 1935 to 1938).

From 1934 to 1936, NEC engineer Akira Nakashima, Claude Shannon and Victor Shestakov introduced switching circuit theory in a series of papers showing that two-valued Boolean algebra, which they discovered independently, can describe the operation of switching circuits.13(https://en.wikipedia.org/wiki/Logic_gate#cite_note-13)[[14]](https://en.wikipedia.org/wiki/Logic_gate#cite_note-14)[[15]](https://en.wikipedia.org/wiki/Logic_gate#cite_note-historical-15)[[16]](https://en.wikipedia.org/wiki/Logic_gate#cite_note-Stankovi%C4%87-Astola_2008-16) Using this property of electrical switches to implement logic is the fundamental concept that underlies

all electronic digital computers. Switching circuit theory became the foundation of digital circuit design, as it became widely known in the electrical engineering community during and after World War II, with theoretical rigor superseding the *ad hoc* methods that had prevailed previously.16(https://en.wikipedia.org/wiki/Logic_gate#cite_note-Stankovi%C4%87-Astola_2008-16)

In 1948, Bardeen and Brattain patented an insulated-gate transistor (IGFET) with an inversion layer. Their concept forms the basis of CMOS technology today.17(https://en.wikipedia.org/wiki/Logic_gate#cite_note-17) In 1957, Frosch and Derick were able to manufacture PMOS and NMOS planar gates.18(https://en.wikipedia.org/wiki/Logic_gate#cite_note-18) Later a team at Bell Labs demonstrated a working MOS with PMOS and NMOS gates.19(https://en.wikipedia.org/wiki/Logic_gate#cite_note-19) Both types were later combined and adapted into complementary MOS (CMOS) logic by Chih-Tang Sah and Frank Wanlass at Fairchild Semiconductor in 1963.20(https://en.wikipedia.org/wiki/Logic_gate#cite_note-computerhistory1963-20)

Symbols



A synchronous 4-bit up/down decade counter symbol (74LS192) in accordance with ANSI/IEEE Std 91-1984 and IEC Publication 60617-12 [missing "C3" at pin 11]

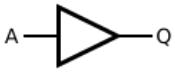
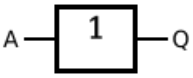
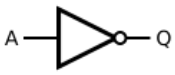
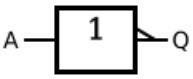
There are two sets of symbols for elementary logic gates in common use, both defined in ANSI/IEEE Std 91-1984 and its supplement ANSI/IEEE Std 91a-1991. The "distinctive shape" set, based on traditional schematics, is used for simple drawings and derives from United States Military Standard MIL-STD-806 of the 1950s and 1960s.21(https://en.wikipedia.org/wiki/Logic_gate#cite_note-21) It is sometimes unofficially described as "military", reflecting its origin. The "rectangular shape" set, based on ANSI Y32.14 and other early industry standards as later refined by IEEE and IEC, has rectangular outlines for all types of gate and allows representation of a much wider range of devices than is possible with the traditional symbols.22(https://en.wikipedia.org/wiki/Logic_gate#cite_note-sdyz001a-22) The IEC standard, IEC 60617-12, has been adopted by other standards, such

as EN 60617-12:1999 in Europe, BS EN 60617-12:1999 in the United Kingdom, and DIN EN 60617-12:1998 in Germany.

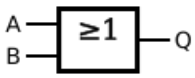
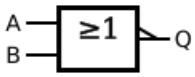
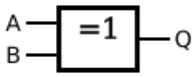
The mutual goal of IEEE Std 91-1984 and IEC 617-12 was to provide a uniform method of describing the complex logic functions of digital circuits with schematic symbols. These functions were more complex than simple AND and OR gates. They could be medium-scale circuits such as a 4-bit counter to a large-scale circuit such as a microprocessor.

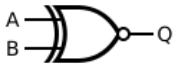
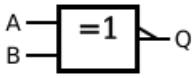
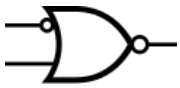
IEC 617-12 and its renumbered successor IEC 60617-12 do not explicitly show the "distinctive shape" symbols, but do not prohibit them.22(https://en.wikipedia.org/wiki/Logic_gate#cite_note-sdyz001a-22) These are, however, shown in ANSI/IEEE Std 91 (and 91a) with this note: "The distinctive-shape symbol is, according to IEC Publication 617, Part 12, not preferred, but is not considered to be in contradiction to that standard." IEC 60617-12 correspondingly contains the note (Section 2.1) "Although non-preferred, the use of other symbols recognized by official national standards, that is distinctive shapes in place of symbols [list of basic gates], shall not be considered to be in contradiction with this standard. Usage of these other symbols in combination to form complex symbols (for example, use as embedded symbols) is discouraged." This compromise was reached between the respective IEEE and IEC working groups to permit the IEEE and IEC standards to be in mutual compliance with one another.

In the 1980s, schematics were the predominant method to design both circuit boards and custom ICs known as gate arrays. Today custom ICs and the field-programmable gate array are typically designed with Hardware Description Languages (HDL) such as Verilog or VHDL.

Type	**Distinctive shape (IEEE Std 91/91a-1991)**	**Rectangular shape (IEEE Std 91/91a-1991) (IEC 60617-12:1997)**	B a A
Single-input gates			
Buffer			A
NOT (inverter)			A

Type	Distinctive shape (IEEE Std 91/91a-1991)	Rectangular shape (IEEE Std 91/91a-1991) (IEC 60617-12:1997)	B a A
In electronics a NOT gate is more commonly called an inverter. The circle on the symbol is called a *bubble* and is used in logic diagrams to indicate a logic negation between the external logic state and the internal logic state (1 to 0 or vice versa). On a circuit diagram it must be accompanied by a statement asserting that the *positive logic convention* or *negative logic convention* is being used (high voltage level = 1 or low voltage level = 1, respectively). The *wedge* is used in circuit diagrams to directly indicate an active-low (low voltage level = 1) input or output without requiring a uniform convention throughout the circuit diagram. This is called *Direct Polarity Indication*. See IEEE Std 91/91A and IEC 60617-12. Both the *bubble* and the *wedge* can be used on distinctive-shape and rectangular-shape symbols on circuit diagrams, depending on the logic convention used. On pure logic diagrams, only the *bubble* is meaningful.			
Conjunction and disjunction			
AND			A A

Type	Distinctive shape (IEEE Std 91/91a-1991)	Rectangular shape (IEEE Std 91/91a-1991) (IEC 60617-12:1997)	B a A
OR			A c
Alternative denial and joint denial			
NAND			A A
NOR			A c
Exclusive or and biconditional			
XOR			A c

Type	Distinctive shape (IEEE Std 91/91a-1991)	Rectangular shape (IEEE Std 91/91a-1991) (IEC 60617-12:1997)	B a A
The output of a two input exclusive-OR is true only when the two input values are *different*, and false if they are equal, regardless of the value. If there are more than two inputs, the output of the distinctive-shape symbol is undefined. The output of the rectangular-shaped symbol is true if the number of true inputs is exactly one or exactly the number following the "=" in the qualifying symbol.			
XNOR			A
Implication and Nonimplication			
IMPLY23(https://en.wikipedia.org/wiki/Logic_gate#cite_note-23)			A
NIMPLY			A

Type	Distinctive shape (IEEE Std 91/91a-1991)	Rectangular shape (IEEE Std 91/91a-1991) (IEC 60617-12:1997)	B a A
IMPLY and NIMPLY are not commutative, meaning that changing the order of the operands may change the result. For instance, A→B$\overline{A} \to \overline{B}$ is false, but A¯→B$\overline{A} \to B$ is true; likewise, A→/B$\overline{A} \not\to \overline{B}$ is true, but A¯→/B$\overline{A} \not\to B$ is false.			

De Morgan equivalent symbols

By use of De Morgan's laws, an *AND* function is identical to an *OR* function with negated inputs and outputs. Likewise, an *OR* function is identical to an *AND* function with negated inputs and outputs. A NAND gate is equivalent to an OR gate with negated inputs, and a NOR gate is equivalent to an AND gate with negated inputs.

This leads to an alternative set of symbols for basic gates that use the opposite core symbol (*AND* or *OR*) but with the inputs and outputs negated. Use of these alternative symbols can make logic circuit diagrams much clearer and help to show accidental connection of an active high output to an active low input or vice versa. Any connection that has logic negations at both ends can be replaced by a negationless connection and a suitable change of gate or vice versa. Any connection that has a negation at one end and no negation at the other can be made easier to interpret by instead using the De Morgan equivalent symbol at either of the two ends. When negation or polarity indicators on both ends of a connection match, there is no logic negation in that path (effectively, bubbles "cancel"), making it easier to follow logic states from one symbol to the next. This is commonly seen in real logic diagrams – thus the reader must not get into the habit of associating the shapes exclusively as OR or AND shapes, but also take into account the bubbles at both inputs and outputs in order to determine the "true" logic function indicated.

A De Morgan symbol can show more clearly a gate's primary logical purpose and the polarity of its nodes that are considered in the "signaled" (active, on) state. Consider the simplified case where a two-input NAND gate is used to drive a motor when either of its inputs are brought low by a switch. The "signaled" state (motor on) occurs when either one OR the other switch is on. Unlike a regular NAND symbol, which suggests AND logic, the De Morgan version, a two negative-input OR gate, correctly shows that OR is of interest. The regular NAND symbol has a bubble at the output and none at the inputs (the opposite of the states that will turn the motor

on), but the De Morgan symbol shows both inputs and output in the polarity that will drive the motor.

De Morgan's theorem is most commonly used to implement logic gates as combinations of only NAND gates, or as combinations of only NOR gates, for economic reasons.

Truth tables

Output comparison of various logic gates:

1-input logic gates
Input	**Output**	
A	Buffer	Inverter
0	0	1
1	1	0

2-input logic gates
Input		**Output**						
A	B	AND	NAND	OR	NOR	XOR	XNOR	IMPLY	NIMPLY
0	0	0	1	0	1	0	1	1	0
0	1	0	1	1	0	1	0	1	0
1	0	0	1	1	0	1	0	0	1
1	1	1	0	1	0	0	1	1	0

Universal logic gates

Further information on the theoretical basis: Functional completeness

Charles Sanders Peirce (during 1880–1881) showed that NOR gates alone (or alternatively NAND gates alone) can be used to reproduce the functions of all the other logic gates, but his work on it was unpublished until 1933.24(https://en.wikipedia.org/wiki/Logic_gate#cite_note-24) The first published proof was by Henry M. Sheffer in 1913, so the NAND logical operation is sometimes called *Sheffer stroke*; the logical NOR is sometimes called *Peirce's arrow*.25(https://en.wikipedia.org/wiki/Logic_gate#cite_note-B%C3%BCningLettmann1999-25) Consequently, these gates are sometimes called *universal logic gates*.26(https://en.wikipedia.org/wiki/Logic_gate#cite_note-Bird2007-26)

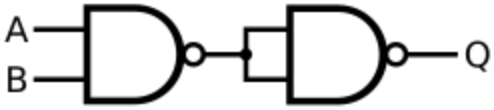
Logic gate constructions from only NAND or only NOR

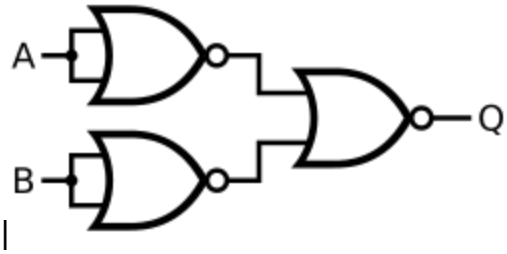
|Type|NAND construction|NOR construction|

|NOT|

|

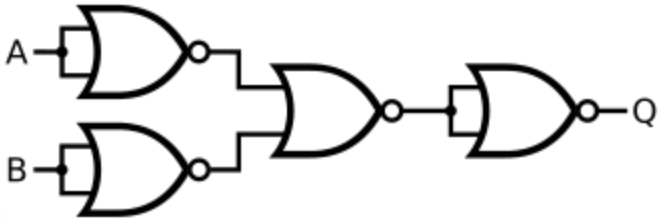
|

|AND| 

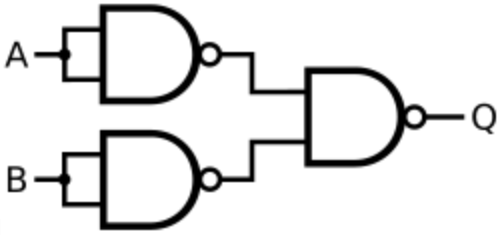
| 

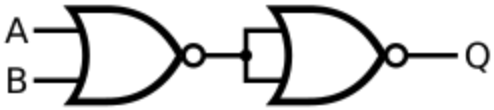
|

|NAND|

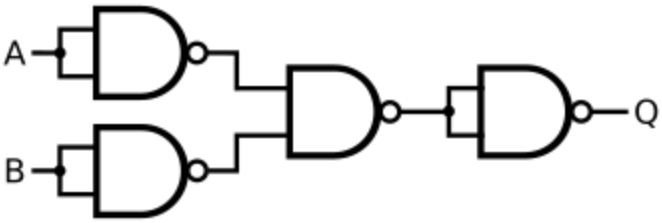
| 

|

|OR| 

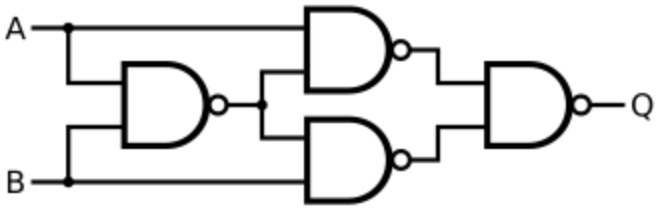
| 

|

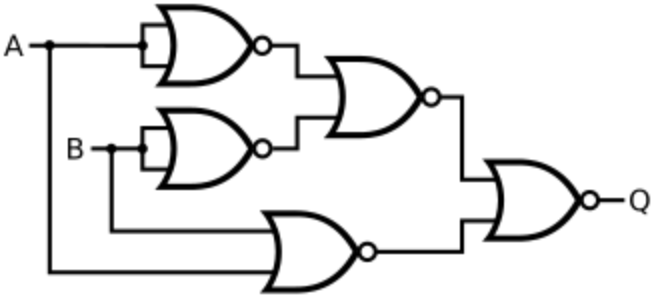
|NOR| 

|

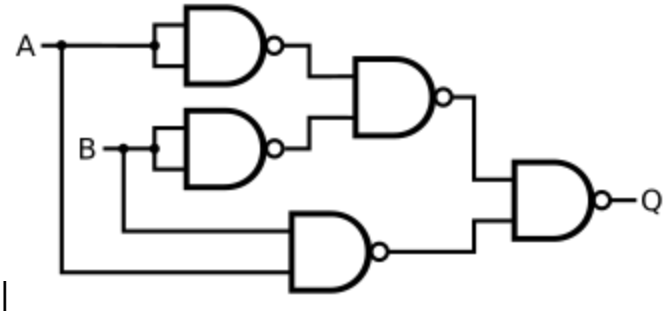
|

|XOR| 

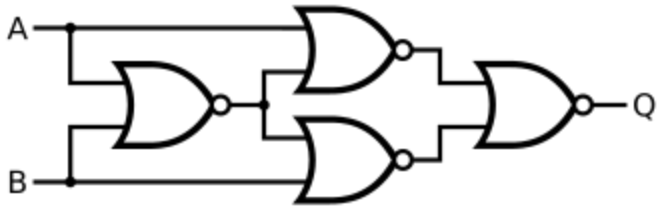
|



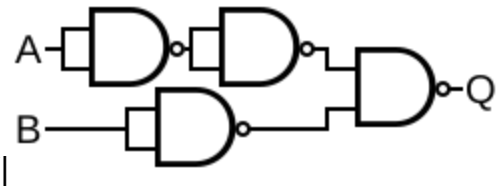
|

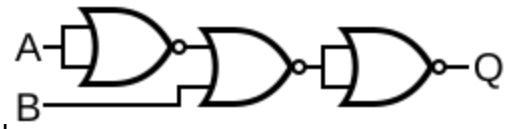


|XNOR| |



|

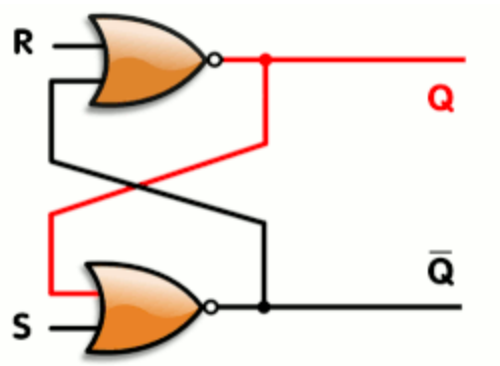




|IMPLY| | |

|NIMPLY|||

Data storage and sequential logic



Animation of how an SR NOR gate latch works

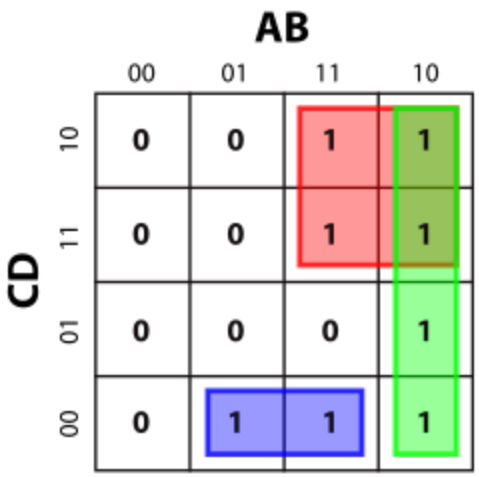
Main article: Sequential logic

Logic gates can also be used to hold a state, allowing data storage. A storage element can be constructed by connecting several gates in a "latch" circuit. Latching circuitry is used in static random-access memory. More complicated designs that use clock signals and that change only on a rising or falling edge of the clock are called edge-triggered "flip-flops". Formally, a flip-flop is called a bistable circuit, because it has two stable states which it can maintain indefinitely. The combination of multiple flip-flops in parallel, used to store a multiple-bit value, is known as a register. When using any of these gate setups the overall system has memory; it is then called a sequential logic system since its output can be influenced by its previous state(s), i.e. by the *sequence* of input states. In contrast, the output from combinational logic is purely a combination of its present inputs, unaffected by the previous input and output states.

These logic circuits are used in computer memory. They vary in performance, based on factors of speed, complexity, and reliability of storage, and many different types of designs are used based on the application.

Karnaugh map

From Wikipedia, the free encyclopedia



An example K-map.

A **Karnaugh map** (**KM** or **K-map**) is a diagram that can be used to simplify a Boolean algebra expression. Maurice Karnaugh introduced the technique in 1953[1](https://en.wikipedia.org/wiki/Karnaugh_map#cite_note-Karnaugh_1953-1)[[2]](https://en.wikipedia.org/wiki/Karnaugh_map#cite_note-Curtis_1962-2) as a refinement of Edward W. Veitch's 1952 **Veitch chart**,[3](https://en.wikipedia.org/wiki/Karnaugh_map#cite_note-Veitch_1952-3)[[4]](https://en.wikipedia.org/wiki/Karnaugh_map#cite_note-Brown_2012-4) which itself was a rediscovery of Allan Marquand's 1881 *logical diagram*[5](https://en.wikipedia.org/wiki/Karnaugh_map#cite_note-Marquand_1881-5)[[6]](https://en.wikipedia.org/wiki/Karnaugh_map#cite_note-Gardner_1958-6) or **Marquand diagram**.[4](https://en.wikipedia.org/wiki/Karnaugh_map#cite_note-Brown_2012-4) They are also known as **Marquand–Veitch diagrams**,[4](https://en.wikipedia.org/wiki/Karnaugh_map#cite_note-Brown_2012-4) **Karnaugh–Veitch (KV) maps**, and (rarely) **Svoboda charts**.[7](https://en.wikipedia.org/wiki/Karnaugh_map#cite_note-Klir_1972-7) An early advance in the history of formal logic methodology, Karnaugh maps remain relevant in the digital age, especially in the fields of logical circuit design and digital engineering.[4](https://en.wikipedia.org/wiki/Karnaugh_map#cite_note-Brown_2012-4)

Definition

A Karnaugh map reduces the need for extensive calculations by taking advantage of humans' pattern-recognition capability.1(https://en.wikipedia.org/wiki/Karnaugh_map#cite_note-Karnaugh_1953-1) It also permits the rapid identification and elimination of potential race conditions.[*clarification needed*]

The required Boolean results are transferred from a truth table onto a two-dimensional grid where, in Karnaugh maps, the cells are ordered in Gray code,8(https://en.wikipedia.org/wiki/Karnaugh_map#cite_note-Wakerly_1994-8)[[4]](https://en.wikipedia.org/wiki/Karnaugh_map#cite_note-Brown_2012-4) and each cell position represents one combination of input conditions. Cells are also known as minterms, while each cell value represents the corresponding output value of the Boolean function. Optimal groups of 1s or 0s are identified, which represent the terms of a canonical form of the logic in the original truth table.9(https://en.wikipedia.org/wiki/Karnaugh_map#cite_note-Belton_1998-9) These terms can be used to write a minimal Boolean expression representing the required logic.

Karnaugh maps are used to simplify real-world logic requirements so that they can be implemented using the minimal number of logic gates. A sum-of-products expression (SOP) can always be implemented using AND gates feeding into an OR gate, and a product-of-sums expression (POS) leads to OR gates feeding an AND gate. The POS expression gives a complement of the function (if F is the function so its complement will be F').10(https://en.wikipedia.org/wiki/Karnaugh_map#cite_note-Dodge_2016-10) Karnaugh maps can also be used to simplify logic expressions in software design. Boolean conditions, as used for example in conditional statements, can get very complicated, which makes the code difficult to read and to maintain. Once minimised, canonical sum-of-products and product-of-sums expressions can be implemented directly using AND and OR logic operators.11(https://en.wikipedia.org/wiki/Karnaugh_map#cite_note-Cook_2012-11)

Example

Karnaugh maps are used to facilitate the simplification of Boolean algebra functions. For example, consider the Boolean function described by the following truth table.

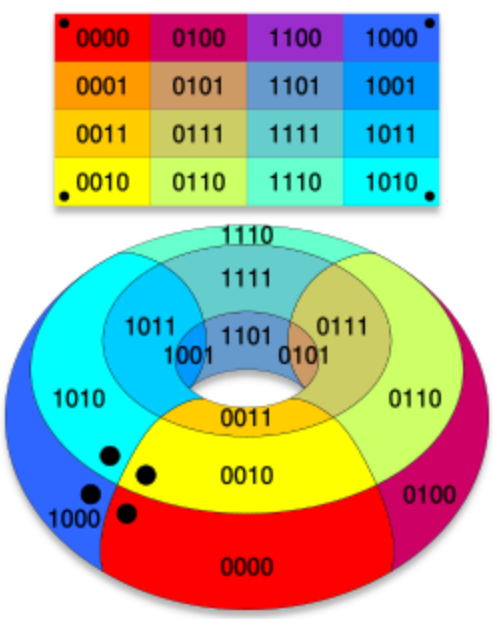
Truth table of a function

A	B	C	D	f(A,B,C,D)$f(A,B,C,D)$
0	0	0	0	0	0
1	0	0	0	1	0
2	0	0	1	0	0
3	0	0	1	1	0

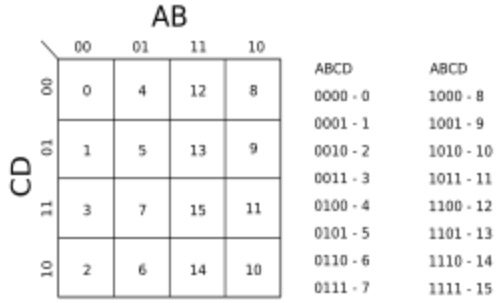
4	0	1	0	0	0
5	0	1	0	1	0
6	0	1	1	0	1
7	0	1	1	1	0
8	1	0	0	0	1
9	1	0	0	1	1
10	1	0	1	0	1
11	1	0	1	1	1
12	1	1	0	0	1
13	1	1	0	1	1
14	1	1	1	0	1
15	1	1	1	1	0

Following are two different notations describing the same function in unsimplified Boolean algebra, using the Boolean variables A, B, C, D and their inverses.

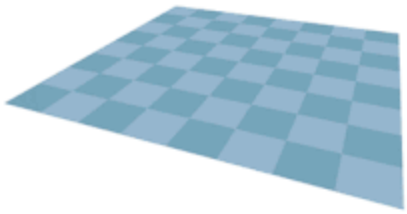
- f(A,B,C,D)=∑mi,i∈{6,8,9,10,11,12,13,14}
  $f(A,B,C,D) = \sum m_i, i \in \{6,8,9,10,11,12,13,14\}$
  where mim_i are the minterms to map (i.e., rows that have output 1 in the truth table).
  $f(A,B,C,D) = \prod M_i, i \in \{0,1,2,3,4,5,7,15\}$
- f(A,B,C,D)=∏Mi,i∈{0,1,2,3,4,5,7,15}
  where MiM_i are the maxterms to map (i.e., rows that have output 0 in the truth table).



K-map drawn on a torus, and in a plane. The dot-marked cells are adjacent.



K-map construction. Instead of the output values (the rightmost values in the truth table), this diagram shows a decimal representation of the input ABCD (the leftmost values in the truth table), therefore it is not a Karnaugh map.



In three dimensions, one can bend a rectangle into a torus.

Construction

In the example above, the four input variables can be combined in 16 different ways, so the truth table has 16 rows, and the Karnaugh map has 16 positions. The Karnaugh map is therefore arranged in a 4 × 4 grid.

The row and column indices (shown across the top and down the left side of the Karnaugh map) are ordered in Gray code rather than binary numerical order. Gray code ensures that only one variable changes between each pair of adjacent cells. Each cell of the completed Karnaugh map contains a binary digit representing the function's output for that combination of inputs.

Grouping

After the Karnaugh map has been constructed, it is used to find one of the simplest possible forms — a canonical form — for the information in the truth table. Adjacent 1s in the Karnaugh map represent opportunities to simplify the expression. The minterms ('minimal terms') for the final expression are found by encircling groups of 1s in the map. Minterm groups must be rectangular and must have an area that is a power of two (i.e., 1, 2, 4, 8...). Minterm rectangles should be as large as possible without containing any 0s. Groups may overlap in order to make each one larger. The optimal groupings in the example below are marked by the green, red and

blue lines, and the red and green groups overlap. The red group is a 2 × 2 square, the green group is a 4 × 1 rectangle, and the overlap area is indicated in brown.

The cells are often denoted by a shorthand which describes the logical value of the inputs that the cell covers. For example, AD would mean a cell which covers the 2x2 area where A and D are true, i.e. the cells numbered 13, 9, 15, 11 in the diagram above. On the other hand, AD would mean the cells where A is true and D is false (that is, D is true).

The grid is toroidally connected, which means that rectangular groups can wrap across the edges (see picture). Cells on the extreme right are actually 'adjacent' to those on the far left, in the sense that the corresponding input values only differ by one bit; similarly, so are those at the very top and those at the bottom. Therefore, AD can be a valid term—it includes cells 12 and 8 at the top, and wraps to the bottom to include cells 10 and 14—as is BD, which includes the four corners.

Solution

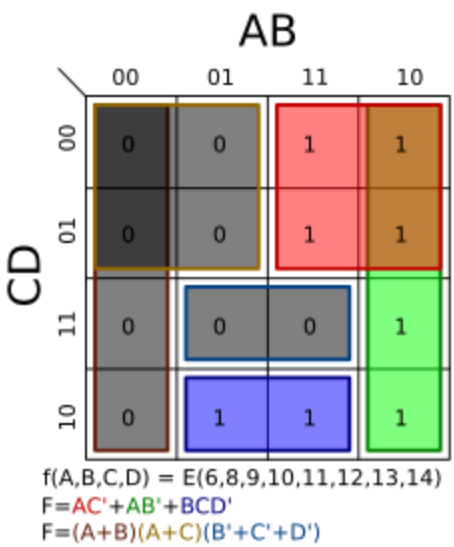


Diagram showing two K-maps. The K-map for the function f(A, B, C, D) is shown as colored rectangles which correspond to minterms. The brown region is an overlap of the red 2×2 square and the green 4×1 rectangle. The K-map for the inverse of f is shown as gray rectangles, which correspond to maxterms.

Once the Karnaugh map has been constructed and the adjacent 1s linked by rectangular and square boxes, the algebraic minterms can be found by examining which variables stay the same within each box.

For the red grouping:

- *A* is the same and is equal to 1 throughout the box, therefore it should be included in the algebraic representation of the red minterm.
- *B* does not maintain the same state (it shifts from 1 to 0), and should therefore be excluded.

- *C* does not change. It is always 0, so its complement, NOT-C, should be included. Thus, C should be included.
- *D* changes, so it is excluded.

Thus the first minterm in the Boolean sum-of-products expression is AC.

For the green grouping, *A* and *B* maintain the same state, while *C* and *D* change. *B* is 0 and has to be negated before it can be included. The second term is therefore AB. Note that it is acceptable that the green grouping overlaps with the red one.

In the same way, the blue grouping gives the term BCD.

The solutions of each grouping are combined: the normal form of the circuit is $AC^{\overline{}}+AB^{\overline{}}+BCD^{\overline{}}$ $A\overline{C}+A\overline{B}+BC\overline{D}$.

Thus the Karnaugh map has guided a simplification of

$f(A,B,C,D)=A^{\overline{}}BCD^{\overline{}}+AB^{\overline{}}C^{\overline{}}D^{\overline{}}+AB^{\overline{}}C^{\overline{}}D+AB^{\overline{}}CD^{\overline{}}+AB^{\overline{}}CD+ABC^{\overline{}}D^{\overline{}}+ABC^{\overline{}}D+ABCD^{\overline{}}=AC^{\overline{}}+$

$$\begin{aligned} f(A,B,C,D) &= \overline{A}BC\overline{D}+A\overline{B}\,\overline{C}\,\overline{D}+A\overline{B}\,\overline{C}D+A\overline{B}C\overline{D}+ \\ &\quad A\overline{B}CD+AB\overline{C}\,\overline{D}+AB\overline{C}D+ABC\overline{D} \\ &= A\overline{C}+A\overline{B}+BC\overline{D} \end{aligned}$$

$AB^{\overline{}}+BCD^{\overline{}}$

It would also have been possible to derive this simplification by carefully applying the axioms of Boolean algebra, but the time it takes to do that grows exponentially with the number of terms.

Inverse

The inverse of a function is solved in the same way by grouping the 0s instead.nb 1(https://en.wikipedia.org/wiki/Karnaugh_map#cite_note-NB_NotNeg-12)

The three terms to cover the inverse are all shown with grey boxes with different colored borders:

- brown: A B
- gold: A C
- blue: BCD

This yields the inverse:

$f(A,B,C,D)^{\overline{}}=A^{\overline{}}B^{\overline{}}+A^{\overline{}}C^{\overline{}}+BCD$ $\overline{f(A,B,C,D)}=\overline{A}\,\overline{B}+\overline{A}\,\overline{C}+BCD$

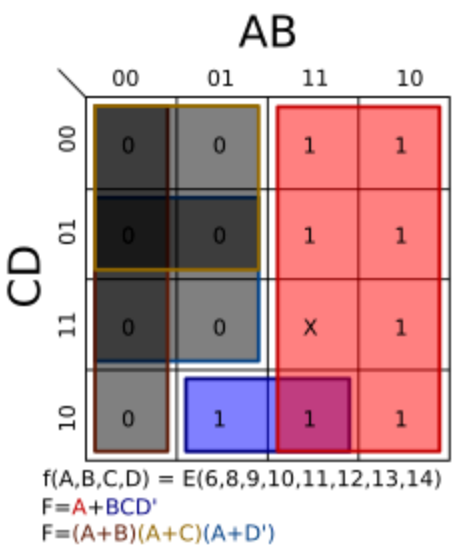
Through the use of De Morgan's laws, the product of sums can be determined:

f(A,B,C,D)=f(A,B,C,D)$\overline{\overline{}}$=A$\overline{B}$+A$\overline{C}$+BCD$\overline{}$=(A$\overline{B}\overline{}$)(A$\overline{C}\overline{}$)(BCD$\overline{}$)=(A+B)(A+C)

$$\begin{aligned} f(A,B,C,D) &= \overline{\overline{f(A,B,C,D)}} \\ &= \overline{\overline{A}\,\overline{B} + \overline{A}\,\overline{C} + BCD} \\ &= \left(\overline{\overline{A}\,\overline{B}}\right)\left(\overline{\overline{A}\,\overline{C}}\right)\left(\overline{BCD}\right) \\ &= (A+B)\,(A+C)\left(\overline{B}+\overline{C}+\overline{D}\right) \end{aligned}$$

(B$\overline{}$+C$\overline{}$+D$\overline{}$)

Don't cares



The value of f(A,B,C,D)$f(A,B,C,D)$ for *ABCD* = 1111 is replaced by a "don't care". This removes the green term completely and allows the red term to be larger. It also allows blue inverse term to shift and become larger

Karnaugh maps also allow easier minimizations of functions whose truth tables include "don't care" conditions. A "don't care" condition is a combination of inputs for which the designer doesn't care what the output is. Therefore, "don't care" conditions can either be included in or excluded from any rectangular group, whichever makes it larger. They are usually indicated on the map with a dash or X.

The example on the right is the same as the example above but with the value of *f*(1,1,1,1) replaced by a "don't care". This allows the red term to expand all the way down and, thus, removes the green term completely.

This yields the new minimum equation:

f(A,B,C,D)=A+BCD$\overline{}$$f(A,B,C,D) = A + BC\overline{D}$

Note that the first term is just A, not AC. In this case, the don't care has dropped a term (the green rectangle); simplified another (the red one); and removed the race hazard (removing the

yellow term as shown in the following section on race hazards).

The inverse case is simplified as follows:

$$\overline{f(A,B,C,D)} = \overline{A}\,\overline{B} + \overline{A}\,\overline{C} + \overline{A}D$$

Through the use of De Morgan's laws, the product of sums can be determined:

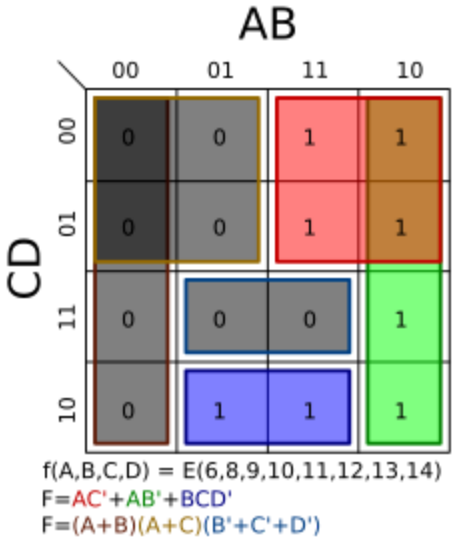
$$f(A,B,C,D) = \overline{\overline{f(A,B,C,D)}} = \overline{\overline{A}\,\overline{B} + \overline{A}\,\overline{C} + \overline{A}\,D} = \left(\overline{\overline{A}\,\overline{B}}\right)\left(\overline{\overline{A}\,\overline{C}}\right)\left(\overline{\overline{A}\,D}\right) = (A+B)\,(A+C)\left(A+\overline{D}\right)$$

Race hazards

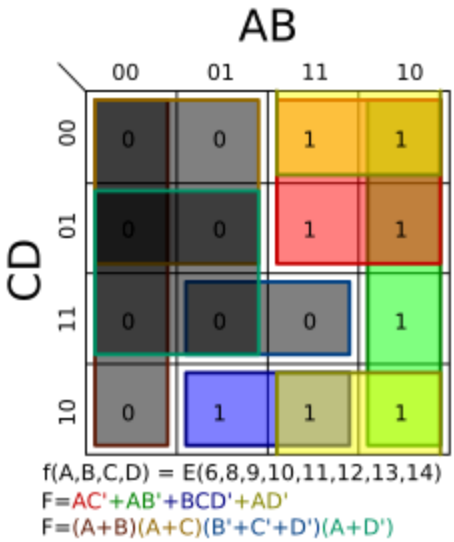
Elimination

Karnaugh maps are useful for detecting and eliminating race conditions. Race hazards are very easy to spot using a Karnaugh map, because a race condition may exist when moving between any pair of adjacent, but disjoint, regions circumscribed on the map. However, because of the nature of Gray coding, *adjacent* has a special definition explained above – we're in fact moving on a torus, rather than a rectangle, wrapping around the top, bottom, and the sides.

- In the example above, a potential race condition exists when *C* is 1 and *D* is 0, *A* is 1, and *B* changes from 1 to 0 (moving from the blue state to the green state). For this case, the output is defined to remain unchanged at 1, but because this transition is not covered by a specific term in the equation, a potential for a *glitch* (a momentary transition of the output to 0) exists.
- There is a second potential glitch in the same example that is more difficult to spot: when *D* is 0 and *A* and *B* are both 1, with C changing from 1 to 0 (moving from the blue state to the red state). In this case the glitch wraps around from the top of the map to the bottom.



Race hazards are present in this diagram.



Above diagram with consensus terms added to avoid race hazards.

Whether glitches will actually occur depends on the physical nature of the implementation, and whether we need to worry about it depends on the application. In clocked logic, it is enough that the logic settles on the desired value in time to meet the timing deadline. In our example, we are not considering clocked logic.

In our case, an additional term of AD¯$A\overline{D}$ would eliminate the potential race hazard, bridging between the green and blue output states or blue and red output states: this is shown as the yellow region (which wraps around from the bottom to the top of the right half) in the adjacent diagram.

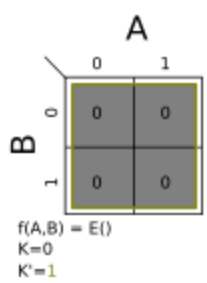
The term is redundant in terms of the static logic of the system, but such redundant, or consensus terms, are often needed to assure race-free dynamic performance.

Similarly, an additional term of A¯D$\overline{A}D$ must be added to the inverse to eliminate another potential race hazard. Applying De Morgan's laws creates another product of sums expression for *f*, but with a new factor of (A+D¯) $\left(A+\overline{D}\right)$.

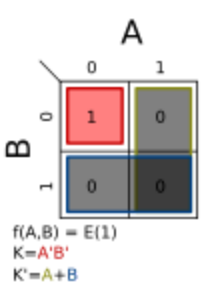
2-variable map examples

The following are all the possible 2-variable, 2 × 2 Karnaugh maps. Listed with each is the minterms as a function of ∑m()$\sum m()$ and the race hazard free (*see* *previous section*) minimum equation. A minterm is defined as an expression that gives the most minimal form of expression of the mapped variables. All possible horizontal and vertical interconnected blocks can be formed. These blocks must be of the size of the powers of 2 (1, 2, 4, 8, 16, 32, ...). These expressions create a minimal logical mapping of the minimal logic variable expressions for the binary expressions to be mapped. Here are all the blocks with one field.

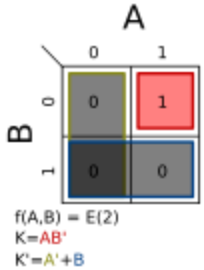
A block can be continued across the bottom, top, left, or right of the chart. That can even wrap beyond the edge of the chart for variable minimization. This is because each logic variable corresponds to each vertical column and horizontal row. A visualization of the k-map can be considered cylindrical. The fields at edges on the left and right are adjacent, and the top and bottom are adjacent. K-Maps for four variables must be depicted as a donut or torus shape. The four corners of the square drawn by the k-map are adjacent. Still more complex maps are needed for 5 variables and more.

- 

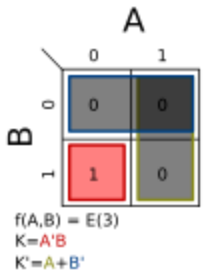
Σ*m*(0); *K* = 0

- 

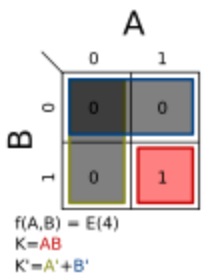

```
Σ_m_(1); _K_ = _A_´_B_´
```

- 

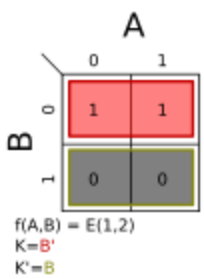

```
Σ_m_(2); _K_ = _AB_´
```

- 

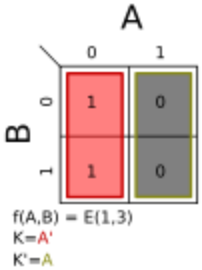

```
Σ_m_(3); _K_ = _A_´_B_
```

- 

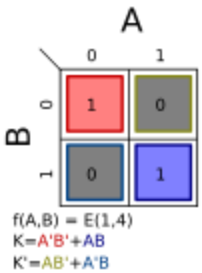

```
Σ_m_(4); _K_ = _AB_
```

- 

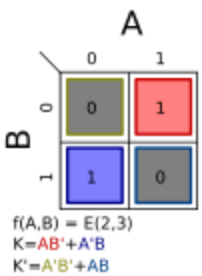

```
Σ_m_(1,2); _K_ = _B_´
```

- 

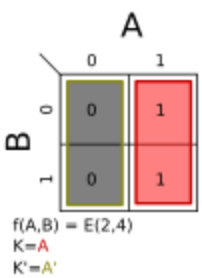

```
Σ_m_(1,3); _K_ = _A_´
```

- 

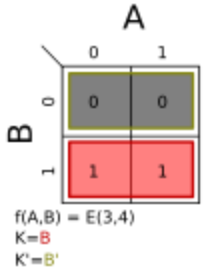

```
Σ_m_(1,4); _K_ = _A_´_B_´ + _AB_
```

- 


```
Σ_m_(2,3); _K_ = _AB_´ + _A_´_B_
```

- 

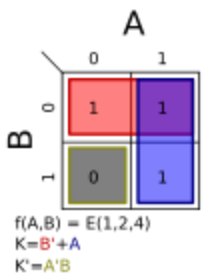

```
Σ_m_(2,4); _K_ = _A_
```

- 

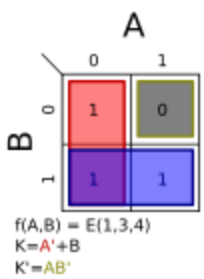

```
Σ_m_(3,4); _K_ = _B_
```

-

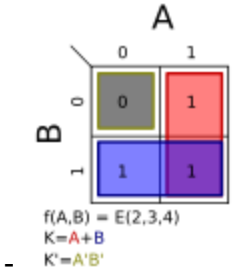

```
Σ_m_(1,2,3); _K_ = _A'_ + _B_´
```

- 

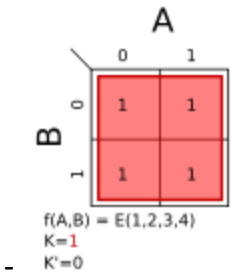

```
Σ_m_(1,2,4); _K_ = _A_ + _B_´
```

- 


```
Σ_m_(1,3,4); _K_ = _A_´ + _B_
```




```
Σ_m_(2,3,4); _K_ = _A_ + _B_
```




```
Σ_m_(1,2,3,4); _K_ = 1
```

Truth table

From Wikipedia, the free encyclopedia

Logical connectives		
NOT	¬A,−A,A¯,~A	$\neg A, -A, \overline{A}, \sim A$
AND	A∧B,A·B,AB,A&B,A&&B	$A \wedge B, A \cdot B, AB, A \,\&\, B, A \,\&\&\, B$
NAND	A∧¯B,A↑B,A|B,A·B¯	$A \overline{\wedge} B, A \uparrow B, A \mid B, \overline{A \cdot B}$
OR	A∨B,A+B,A|B,A‖B	$A \vee B, A + B, A \mid B, A \parallel B$
NOR	A∨¯B,A↓B,A+B¯	$A \overline{\vee} B, A \downarrow B, \overline{A + B}$
XNOR	A⊙B,A∨¯B¯	$A \odot B, \overline{A \overline{\vee} B}$
└ equivalent	A≡B,A⇔B,A⇆B	$A \equiv B, A \Leftrightarrow B, A \leftrightarrows B$
XOR	A∨B,A⊕B	$A \veebar B, A \oplus B$
└ nonequivalent	A≢B,A⇔/B,A↔/B	$A \not\equiv B, A \not\Leftrightarrow B, A \not\leftrightarrow B$
implies	A⇒B,A⊃B,A→B	$A \Rightarrow B, A \supset B, A \to B$
nonimplication (NIMPLY)	A⇏B,A⊅B,A↛B	$A \not\Rightarrow B, A \not\supset B, A \not\to B$

Logical connectives
|[converse](https://en.wikipedia.org/wiki/Converse(logic) "Converse (logic)")|A⇐B,A⊂B,A←B $A \Leftarrow B, A \subset B, A \leftarrow B$|
|converse nonimplication|A⇍B,A⊄B,A↚B $A \nLeftarrow B, A \not\subset B, A \nleftarrow B$|
Related concepts
- Propositional calculus - Predicate logic - Boolean algebra - Truth table - Truth function - Boolean function - Functional completeness - Scope (logic)
Applications
- Digital logic - Programming languages - Mathematical logic - Philosophy of logic
Category
- v - t - e

A **truth table** is a mathematical table used in logic—specifically in connection with Boolean algebra, Boolean functions, and propositional calculus—which sets out the functional values of logical expressions on each of their functional arguments, that is, for each combination of values taken by their logical variables.1(https://en.wikipedia.org/wiki/Truth_table#cite_note-1) In particular, truth tables can be used to show whether a propositional expression is true for all legitimate input values, that is, logically valid.

A truth table has one column for each input variable (for example, A and B), and one final column showing the result of the logical operation that the table represents (for example, A XOR B). Each row of the truth table contains one possible configuration of the input variables (for instance, A=true, B=false), and the result of the operation for those values.

A proposition's truth table is a graphical representation of its truth function. The truth function can be more useful for mathematical purposes, although the same information is encoded in both.

Ludwig Wittgenstein is generally credited with inventing and popularizing the truth table in his *Tractatus Logico-Philosophicus*, which was completed in 1918 and published in

1921.2(https://en.wikipedia.org/wiki/Truth_table#cite_note-2) Such a system was also independently proposed in 1921 by Emil Leon Post.3(https://en.wikipedia.org/wiki/Truth_table#cite_note-3)

History

Irving Anellis's research shows that C.S. Peirce appears to be the earliest logician (in 1883) to devise a truth table matrix.4(https://en.wikipedia.org/wiki/Truth_table#cite_note-Peirce-4)

From the summary of Anellis's paper:4(https://en.wikipedia.org/wiki/Truth_table#cite_note-Peirce-4)

> In 1997, John Shosky discovered, on the verso of a page of the typed transcript of Bertrand Russell's 1912 lecture on "The Philosophy of Logical Atomism" truth table matrices. The matrix for negation is Russell's, alongside of which is the matrix for material implication in the hand of Ludwig Wittgenstein. It is shown that an unpublished manuscript identified as composed by Peirce in 1893 includes a truth table matrix that is equivalent to the matrix for material implication discovered by John Shosky. An unpublished manuscript by Peirce identified as having been composed in 1883–84 in connection with the composition of Peirce's "On the Algebra of Logic: A Contribution to the Philosophy of Notation" that appeared in the *American Journal of Mathematics* in 1885 includes an example of an indirect truth table for the conditional.

Applications

Truth tables can be used to prove many other logical equivalences. For example, consider the following truth table:

(p→q)≡(¬p∨q)$(p \to q) \equiv (\neg p \vee q)$
pp	qq	¬p$\neg p$	¬p∨q$\neg p \vee q$	p→q$p \to q$
T	T	F	T	T
T	F	F	F	F
F	T	T	T	T
F	F	T	T	T

This demonstrates the fact that p→q$p \to q$ is logically equivalent to ¬p∨q$\neg p \vee q$.

Truth table for logic gates

Here is a truth table that gives definitions of each of the 6 possible 2-input logic gate functions of two Boolean variables P and Q:

PP	QQ	P∧Q$P \wedge Q$	P∨Q$P \vee Q$	P↑Q$P \uparrow Q$
T	T	T	T	F
T	F	F	T	T
F	T	F	T	T
F	F	F	F	T
Name (function)		AND (conjunction)	OR (disjunction)	NAND (non-conjunction)
where T *means* **true** *and* F *means* **false**				

Condensed truth tables for binary operators

For binary operators, a condensed form of truth table is also used, where the row headings and the column headings specify the operands and the table cells specify the result. For example, Boolean logic uses this condensed truth table notation:

	|∧|T|F| |---|---|---| |T|T|F| |F|F|F|		|∨|T|F| |---|---|---| |T|T|T| |F|T|F|

This notation is useful especially if the operations are commutative, although one can additionally specify that the rows are the first operand and the columns are the second operand. This condensed notation is particularly useful in discussing multi-valued extensions of logic, as it significantly cuts down on combinatoric explosion of the number of rows otherwise needed. It also provides for quickly recognizable characteristic "shape" of the distribution of the values in the table which can assist the reader in grasping the rules more quickly.

Truth tables in digital logic

Truth tables are also used to specify the function of hardware look-up tables (LUTs) in digital logic circuitry. For an n-input LUT, the truth table will have 2n2^n values (or rows in the above tabular format), completely specifying a Boolean function for the LUT. By representing each Boolean value as a bit in a binary number, truth table values can be efficiently encoded as integer values in electronic design automation (EDA) software. For example, a 32-bit integer can encode the truth table for a LUT with up to 5 inputs.

When using an integer representation of a truth table, the output value of the LUT can be obtained by calculating a bit index *k* based on the input values of the LUT, in which case the LUT's output value is the *k_th bit of the integer. For example, to evaluate the output value of a LUT given an* [array](#) *of _n* Boolean input values, the bit index of the truth table's output value can be computed as follows: if the *i_th input is true, let Vi=1![{\displaystyle V*{i}=1}] (https://wikimedia.org/api/rest_v1/media/math/render/svg/af98ff0a21a287cb51f7be55e5d5c06c2421f3c8), else let Vi=0$V_i = 0$. Then the *k_th bit of the binary representation of the truth table is the LUT's output value, wherek=V0×20+V1×21+V2×22+⋯+Vn−1×2n−1.![{\displaystyle k=V*{0}\times 2^{0}+V*{1}\times 2^{1}+V*{2}\times 2^{2}+\dots +V_{n-1}\times 2^{n-1}.}] (https://wikimedia.org/api/rest_v1/media/math/render/svg/52845d4414ae4c75a30a9b7cd8a43a5d16632782)

Truth tables are a simple and straightforward way to encode Boolean functions, however given the exponential growth in size as the number of inputs increase, they are not suitable for functions with a large number of inputs. Other representations which are more memory efficient are text equations and binary decision diagrams.

Applications of truth tables in digital electronics

In digital electronics and computer science (fields of applied logic engineering and mathematics), truth tables can be used to reduce basic Boolean operations to simple correlations of inputs to outputs, without the use of logic gates or code. For example, a binary addition can be represented with the truth table:

Binary addition
A	B	C	R
T	T	T	F
T	F	F	T
F	T	F	T
F	F	F	F

where A is the first operand, B is the second operand, C is the carry digit, and R is the result.

This truth table is read left to right:

- Value pair (A, B) equals value pair (C, R).
- Or for this example, A plus B equal result R, with the Carry C.

This table does not describe the logic operations necessary to implement this operation, rather it simply specifies the function of inputs to output values.

With respect to the result, this example may be arithmetically viewed as modulo 2 binary addition, and as logically equivalent to the exclusive-or (exclusive disjunction) binary logic operation.

In this case it can be used for only very simple inputs and outputs, such as 1s and 0s. However, if the number of types of values one can have on the inputs increases, the size of the truth table will increase.

For instance, in an addition operation, one needs two operands, A and B. Each can have one of two values, zero or one. The number of combinations of these two values is 2 × 2, or four. So the result is four possible outputs of C and R. If one were to use base 3, the size would increase to 3 × 3, or nine possible outputs.

The first "addition" example above is called a half-adder. A full-adder is when the carry from the previous operation is provided as input to the next adder. Thus, a truth table of eight rows would be needed to describe a full adder's logic:

A B C* | C R
0 0 0 | 0 0
0 1 0 | 0 1
1 0 0 | 0 1
1 1 0 | 1 0
0 0 1 | 0 1
0 1 1 | 1 0
1 0 1 | 1 0
1 1 1 | 1 1

Same as previous, but..
C* = Carry from previous adder

Methods of writing truth tables

Regarding the *guide columns*5*(https://en.wikipedia.org/wiki/Truth_table#cite_note-:0-5)* to the left of a table, which represent propositional variables, different authors have different recommendations about how to fill them in, although this is of no logical significance.6(https://en.wikipedia.org/wiki/Truth_table#cite_note-:13-6)

Alternating method

Lee Archie, a professor at Lander University, recommends this procedure, which is commonly followed in published truth-tables:

1. Write out the number of variables (corresponding to the number of statements) in alphabetical order.

2. The number of lines needed is 2*n* where n is the number of variables. (E. g., with three variables, 23 = 8).
3. Start in the right-hand column and alternate **T**'s and **F**'s until you run out of lines.
4. Then move left to the next column and alternate pairs of **T**'s and **F**'s until you run out of lines.
5. Then continue to the next left-hand column and double the numbers of **T**'s and **F**'s until completed.5(https://en.wikipedia.org/wiki/Truth_table#cite_note-:0-5)

This method results in truth-tables such as the following table for $P \to (Q \vee R \to (R \to \neg P))$, produced by Stephen Cole Kleene:7(https://en.wikipedia.org/wiki/Truth_table#cite_note-7)

PP	QQ	RR	P→(Q∨R→(R→¬P))$P \to (Q \vee R \to (R \to \neg P))$
T	T	T	F
T	T	F	T
T	F	T	F
T	F	F	T
F	T	T	T
F	T	F	T
F	F	T	T
F	F	F	T

Combinatorial method

Colin Howson, on the other hand, believes that "it is a good practical rule" to do the following:

> to start with all Ts, then all the ways (three) two Ts can be combined with one F, then all the ways (three) one T can be combined with two Fs, and then finish with all Fs. If a compound is built up from n distinct sentence letters, its truth table will have 2n rows, since there are two ways of assigning T or F to the first letter, and for each of these there will be two ways of assigning T or F to the second, and for each of these there will be two ways of assigning T or F to the third, and so on, giving 2.2.2. …, n times, which is equal to 2n.6(https://en.wikipedia.org/wiki/Truth_table#cite_note-:13-6)

This results in truth tables like this table "showing that $(A \to C) \wedge (B \to C)$ and $(A \vee B) \to C$ are truth-functionally equivalent", modeled after a table produced by Howson:6(https://en.wikipedia.org/wiki/Truth_table#cite_note-:13-6)

AA	BB	CC	(A→C)∧(B→C)$(A \to C) \wedge (B \to C)$	(A∨B)→C$(A \vee B) \to C$
T	T	T	T	T
T	T	F	F	F
T	F	T	T	T
F	T	T	T	T
F	F	T	T	T
F	T	F	F	F
T	F	F	F	F
F	F	F	T	T

Size of truth tables

If there are *n* input variables then there are 2*n* possible combinations of their truth values. A given function may produce true or false for each combination so the number of different functions of *n* variables is the double exponential 22*n*.

n	***2n***	***22n***
0	1	2
1	2	4
2	4	16
3	8	256
4	16	65,536
5	32	4,294,967,296
6	64	18,446,744,073,709,551,616
7	128	340,282,366,920,938,463,463,374,607,431,768,211,456
8	256	115,792,089,237,316,195,423,570,985,008,687,907,853,269,984,665,640,564,(

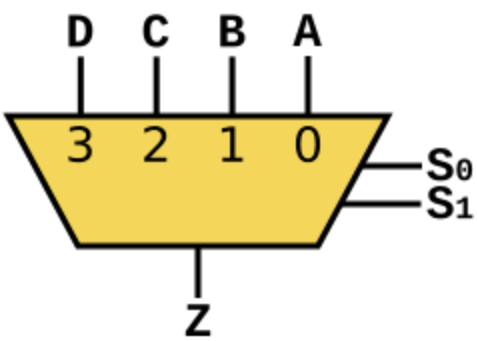
Truth tables for functions of three or more variables are rarely given.

Function Tables

It can be useful to have the output of a truth table expressed as a function of some variable values, instead of just a literal truth or false value. These may be called "function tables" to differentiate them from the more general "truth tables".8(https://en.wikipedia.org/wiki/Truth_table#cite_note-8) For example, one value, G, may be used with an XOR gate to conditionally invert another value, X. In other words, when G is false, the output is X, and when G is true, the output is ¬X$\neg X$. The function table for this would look like:

GG	G ↔ X$G \leftrightarrow X$
F	XX
T	¬X$\neg X$

Similarly, a 4-to-1 multiplexer with select inputs S0S_0 and S1S_1, data inputs A, B, C and D, and output Z (as displayed in the image) would have this function table:



4-to-1 multiplexer

S1S_1	S0S_0	Z
F	F	A
F	T	B
T	F	C
T	T	D

Sentential operator truth tables

Overview table

Here is an extended truth table giving definitions of all sixteen possible truth functions of two Boolean variables ***p*** and ***q***:note 1(https://en.wikipedia.org/wiki/Truth_table#cite_note-9)

pp	qq		⊥$\bot$	p↓q $p \downarrow q$	p ←/q $p \nleftarrow q$	¬p $\neg p$	p →/q $p \nrightarrow q$	¬q $\neg q$	p ↔/q $p \nleftrightarrow q$	p↑q $p \uparrow q$	p∧q $p \wedge q$	p ↔ $p \leftrightarrow$
T	T		F	F	F	F	F	F	F	F	T	T
T	F		F	F	F	F	T	T	T	T	F	F
F	T		F	F	T	T	F	F	T	T	F	F
F	F		F	T	F	T	F	T	F	T	F	T
Com			✓	✓					✓	✓	✓	✓
Assoc			✓						✓		✓	✓
Adj			⊥$\bot$	p↓q $p \downarrow q$	p →/q $p \nrightarrow q$	¬q $\neg q$	p ←/q $p \nleftarrow q$	¬p $\neg p$	p ↔/q $p \nleftrightarrow q$	p↑q $p \uparrow q$	p∧q $p \wedge q$	p ↔ $p \leftrightarrow$
Neg			⊤$\top$	p∨q $p \vee q$	p ← q $p \leftarrow q$	pp	p → q $p \rightarrow q$	qq	p ↔ q $p \leftrightarrow q$	p∧q $p \wedge q$	p↑q $p \uparrow q$	p ↔ $p \leftrightarrow$
Dual			⊤$\top$	p↑q $p \uparrow q$	p → q $p \rightarrow q$	¬p $\neg p$	p ← q $p \leftarrow q$	¬q $\neg q$	p ↔ q $p \leftrightarrow q$	p↓q $p \downarrow q$	p∨q $p \vee q$	p ↔ $p \leftrightarrow$
L id					F				F		T	T
R id							F		F		T	T

where

T = true.

F = false.

The **Com** row indicates whether an operator, **op**, is commutative – *P* op *Q* = *Q* op *P*.

The **Assoc** row indicates whether an operator, **op**, is associative – (*P* op *Q*) op *R* = *P* op (*Q* op *R*).

The **Adj** row shows the operator **op2** such that *P* op *Q* = *Q* op2 *P*.

The **Neg** row shows the operator **op2** such that *P* op *Q* = ¬(*P* op2 *Q*).

The **Dual** row shows the dual operation obtained by interchanging T with F, and AND with OR.

The **L id** row shows the operator's left identities if it has any values I such that *I* op *Q* = *Q*.

The **R id** row shows the operator's right identities if it has any values I such that *P* op *I* = *P*.note 2(https://en.wikipedia.org/wiki/Truth_table#cite_note-10)

Wittgenstein table

In proposition 5.101 of the *Tractatus Logico-Philosophicus*,9(https://en.wikipedia.org/wiki/Truth_table#cite_note-tlp5.101-11) Wittgenstein listed the table above as follows:

	Truthvalues		**Operator**		**Operation name**	**Tractatus**note 3**(**https://en.wikipedia.o different_mapping-12**)**
0	(F F F F)(p, q)	⊥	false	**Opq**	Contradiction	p and not p; and q and
1	(F F F T)(p, q)	NOR	**p ↓ q**	**Xpq**	Logical NOR	neither *p* nor *q*
2	(F F T F)(p, q)	←/	**p ←/ q**	**Mpq**	Converse nonimplication	*q* and not *p*
3	(F F T T)(p, q)	**¬p**, **~p**	**¬p**	**Np**, **Fpq**	Negation	not *p*
4	(F T F F)(p, q)	→/	**p →/ q**	**Lpq**	Material nonimplication	*p* and not *q*
5	(F T F T)(p, q)	**¬q**, **~q**	**¬q**	**Nq**, **Gpq**	Negation	not *q*
6	(F T T F)(p, q)	XOR	**p ⊕ q**	**Jpq**	Exclusive disjunction	*p* or *q*, but not both
7	(F T T T)(p, q)	NAND	**p ↑ q**	**Dpq**	Logical NAND	not both *p* and *q*
8	(T F F F)(p, q)	AND	**p ∧ q**	**Kpq**	Logical conjunction	*p* and *q*
9	(T F F T)(p, q)	XNOR	**p** iff **q**	**Epq**	Logical biconditional	if *p* then *q*; and if *q* the
10	(T F T F)(p, q)	**q**	**q**	**Hpq**	Projection function	*q*
11	(T F T T)(p, q)	**p → q**	if **p** then **q**	**Cpq**	Material implication	if *p* then *q*
12	(T T F F)(p, q)	**p**	**p**	**Ipq**	Projection function	*p*
13	(T T F T)(p, q)	**p ← q**	if **q** then **p**	**Bpq**	Converse implication	if *q* then *p*
14	(T T T F)(p, q)	OR	**p ∨ q**	**Apq**	Logical disjunction	*p* or *q*

	Truthvalues		Operator		Operation name	Tractatusnote 3(https://en.wikipedia.o different_mapping-12)
15	(T T T T)(p, q)	⊤	true	**Vpq**	Tautology	if p then p; and if q the

The truth table represented by each row is obtained by appending the sequence given in **Truthvalues**row to the tablenote 3(https://en.wikipedia.org/wiki/Truth_table#cite_note-different_mapping-12)

p	T	T	F	F
q	T	F	T	F

For example, the table

p	T	T	F	F
q	T	F	T	F
11	T	F	T	T

represents the truth table for Material implication. Logical operators can also be visualized using Venn diagrams.

Nullary operations

There are 2 nullary operations:

- Always true
- Never true, unary *falsum*

Logical true

The output value is always true, because this operator has zero operands and therefore no input values

p	***T***
T	T
F	T

Logical false

The output value is never true: that is, always false, because this operator has zero operands and therefore no input values

p	***F***
T	F
F	F

Unary operations

There are 2 unary operations:

- Unary *identity*
- Unary *negation*

Logical identity

Logical identity is an operation on one logical value p, for which the output value remains p.

The truth table for the logical identity operator is as follows:

p	***p***
T	T
F	F

Logical negation

Logical negation is an operation on one logical value, typically the value of a proposition, that produces a value of *true* if its operand is false and a value of *false* if its operand is true.

The truth table for **NOT p** (also written as **¬p**, **Np**, **Fpq**, or **~p**) is as follows:

p	***¬p***
T	F
F	T

Binary operations

There are 16 possible truth functions of two binary variables, each operator has its own name.

Logical conjunction (AND)

Logical conjunction is an operation on two logical values, typically the values of two propositions, that produces a value of *true* if both of its operands are true.

The truth table for **p AND q** (also written as **p ∧ q**, **Kpq**, **p & q**, or **p ·· q**) is as follows:

p	*q*	*p* ∧ *q*
T	T	T
T	F	F
F	T	F
F	F	F

In ordinary language terms, if both *p* and *q* are true, then the conjunction *p* ∧ *q* is true. For all other assignments of logical values to *p* and to *q* the conjunction *p* ∧ *q* is false.

It can also be said that if *p*, then *p* ∧ *q* is *q*, otherwise *p* ∧ *q* is *p*.

Logical disjunction (OR)

Logical disjunction is an operation on two logical values, typically the values of two propositions, that produces a value of *true* if at least one of its operands is true.

The truth table for **p OR q** (also written as **p ∨ q**, **Apq**, **p || q**, or **p + q**) is as follows:

p	*q*	*p* ∨ *q*
T	T	T
T	F	T
F	T	T
F	F	F

Stated in English, if *p*, then *p* ∨ *q* is *p*, otherwise *p* ∨ *q* is *q*.

Logical implication

Logical implication and the material conditional are both associated with an operation on two logical values, typically the values of two propositions, which produces a value of *false* if the first operand is true and the second operand is false, and a value of *true* otherwise.

The truth table associated with the logical implication **p implies q** (symbolized as **p ⇒ q**, or more rarely **Cpq**) is as follows:

p	*q*	*p ⇒ q*
T	T	T
T	F	F
F	T	T
F	F	T

The truth table associated with the material conditional **if p then q** (symbolized as **p → q**) is as follows:

p	*q*	*p → q*
T	T	T
T	F	F
F	T	T
F	F	T

p ⇒ q and **p → q** are equivalent to **¬p ∨ q**.

Logical equality

Logical equality (also known as biconditional or exclusive nor) is an operation on two logical values, typically the values of two propositions, that produces a value of *true* if both operands are false or both operands are true.

The truth table for **p XNOR q** (also written as **p ↔ q**, **Epq**, **p = q**, or **p ≡ q**) is as follows:

p	*q*	*p ↔ q*
T	T	T
T	F	F
F	T	F
F	F	T

So p EQ q is true if p and q have the same truth value (both true or both false), and false if they have different truth values.

Exclusive disjunction

Exclusive disjunction is an operation on two logical values, typically the values of two propositions, that produces a value of *true* if one but not both of its operands is true.

The truth table for **p XOR q** (also written as **Jpq**, or **p ⊕ q**) is as follows:

p	***q***	**p ⊕ q**
T	T	F
T	F	T
F	T	T
F	F	F

For two propositions, **XOR** can also be written as (p ∧ ¬q) ∨ (¬p ∧ q).

Logical NAND

The logical NAND is an operation on two logical values, typically the values of two propositions, that produces a value of *false* if both of its operands are true. In other words, it produces a value of *true* if at least one of its operands is false.

The truth table for **p NAND q** (also written as **p ↑ q**, **Dpq**, or **p | q**) is as follows:

p	***q***	***p ↑ q***
T	T	F
T	F	T
F	T	T
F	F	T

It is frequently useful to express a logical operation as a compound operation, that is, as an operation that is built up or composed from other operations. Many such compositions are possible, depending on the operations that are taken as basic or "primitive" and the operations that are taken as composite or "derivative".

In the case of logical NAND, it is clearly expressible as a compound of NOT and AND.

The negation of a conjunction: ¬(p ∧ q), and the disjunction of negations: (¬p) ∨ (¬q) can be tabulated as follows:

p	q	$p \wedge q$	$\neg(p \wedge q)$	$\neg p$	$\neg q$	$(\neg p) \vee (\neg q)$
T	T	T	F	F	F	F
T	F	F	T	F	T	T
F	T	F	T	T	F	T
F	F	F	T	T	T	T

Logical NOR

The logical NOR is an operation on two logical values, typically the values of two propositions, that produces a value of *true* if both of its operands are false. In other words, it produces a value of *false* if at least one of its operands is true. ↓ is also known as the Peirce arrow after its inventor, Charles Sanders Peirce, and is a Sole sufficient operator.

The truth table for **p NOR q** (also written as **p ↓ q**, or **Xpq**) is as follows:

p	q	$p \downarrow q$
T	T	F
T	F	F
F	T	F
F	F	T

The negation of a disjunction $\neg(p \vee q)$, and the conjunction of negations $(\neg p) \wedge (\neg q)$ can be tabulated as follows:

p	q	$p \vee q$	$\neg(p \vee q)$	$\neg p$	$\neg q$	$(\neg p) \wedge (\neg q)$
T	T	T	F	F	F	F
T	F	T	F	F	T	F
F	T	T	F	T	F	F
F	F	F	T	T	T	T

Inspection of the tabular derivations for NAND and NOR, under each assignment of logical values to the functional arguments p and q, produces the identical patterns of functional values for $\neg(p \wedge q)$ as for $(\neg p) \vee (\neg q)$, and for $\neg(p \vee q)$ as for $(\neg p) \wedge (\neg q)$. Thus the first and second expressions in each pair are logically equivalent, and may be substituted for each other in all contexts that pertain solely to their logical values.

This equivalence is one of De Morgan's laws.